

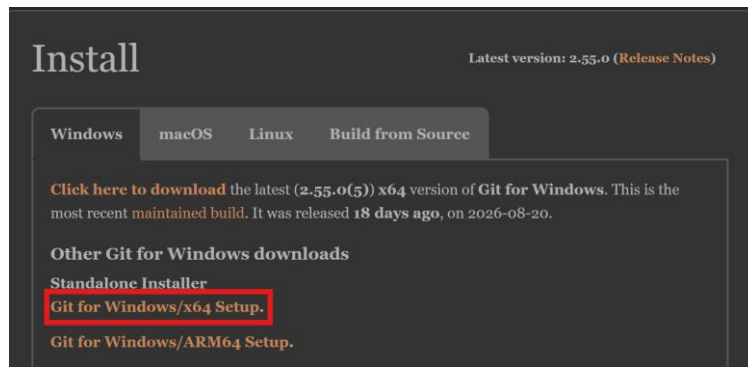
Git GitHub 공동 저장소 실습

VS Code의 소스 제어 화면을 중심으로 이슈 생성부터 PR 리뷰와 머지까지 협업 과정을 실습합니다. VS Code 설치와 GitHub 계정 준비가 끝난 상태에서 시작합니다.

진행 순서 초대 수락 → 이슈 → 복제 → 브랜치 → 파일 수정 → 커밋과 푸시 → PR → 리뷰 → 머지 → main 동기화
본문의 team00, gildong, #12는 예시입니다. 자신의 팀 폴더, 이름, 이슈 번호로 바꿔 진행합니다. 캡처 속 이름과 파일은 화면 조작을 설명하기 위한 예시입니다.

0 Git 설치 확인

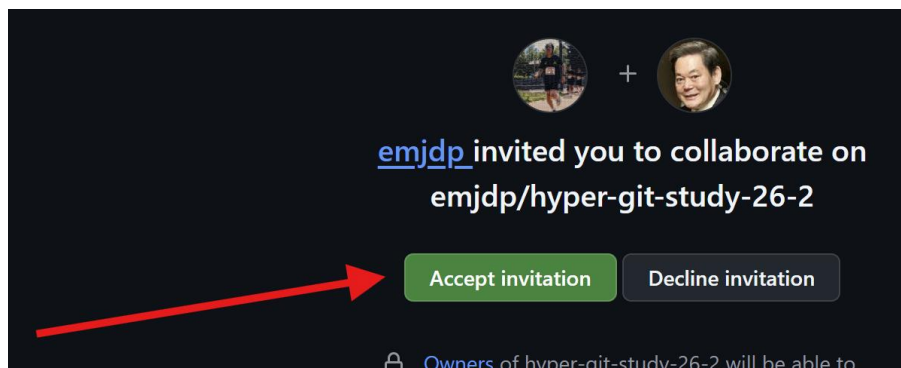
Git이 설치되어 있다면 다음 단계로 넘어갑니다. 아직 설치하지 않았다면 [Git for Windows](#)에서 설치한 뒤 VS Code를 다시 실행합니다.



Git이 없다면 Git for Windows를 설치합니다.

1 저장소 초대 수락

1. 신청 폼에 입력한 계정으로 GitHub에 로그인합니다.
2. [저장소 초대 페이지](#)에서 **Accept invitation**을 누릅니다. 초대를 수락하면 실습 저장소의 쓰기 권한이 추가됩니다.

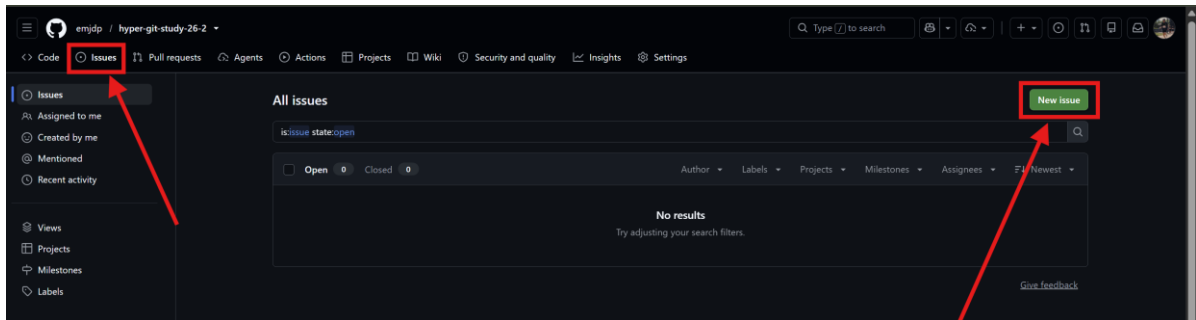


초대를 받은 GitHub 계정으로 로그인한 상태에서 수락합니다.

실습 저장소 [emjdp/hyper-git-study-26-2](#)

1 이슈 만들기

1. 실습 저장소의 [Issues](#) 에서 **New issue** 를 누릅니다.

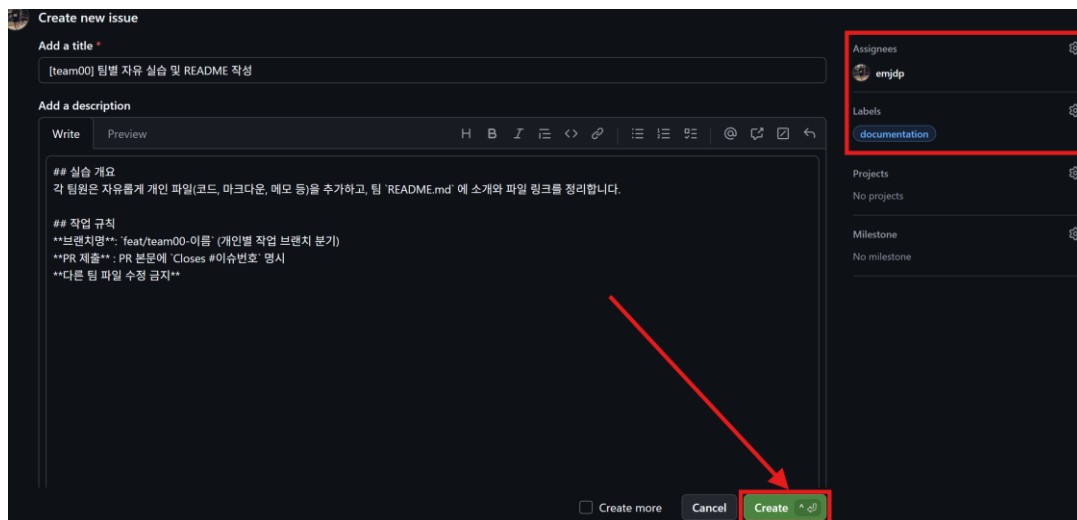


Issues 탭에서 New issue 를 선택합니다.

2. 제목과 본문에 이번에 할 작업을 적습니다. 아래 예시를 팀 활동에 맞게 바꿉니다.

제목: [team00] 팀별 자유 실습 및 README 작성

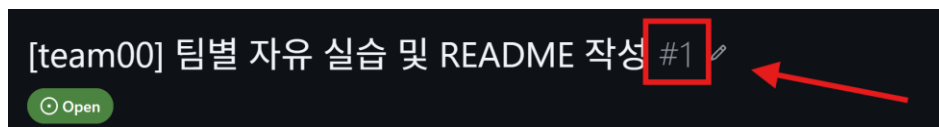
본문: 팀 활동 파일을 추가하고 team00/README.md 에 링크합니다.



이슈에 작업할 내용을 적습니다.

3. **Assignees** 에서 자신을 담당자로 지정하고, 작업에 맞는 **Labels** 를 선택합니다.

4. **Create** 를 눌러 이슈를 생성합니다. 생성 후 표시되는 #12 같은 번호는 PR 에서 이슈를 연결할 때 사용합니다.



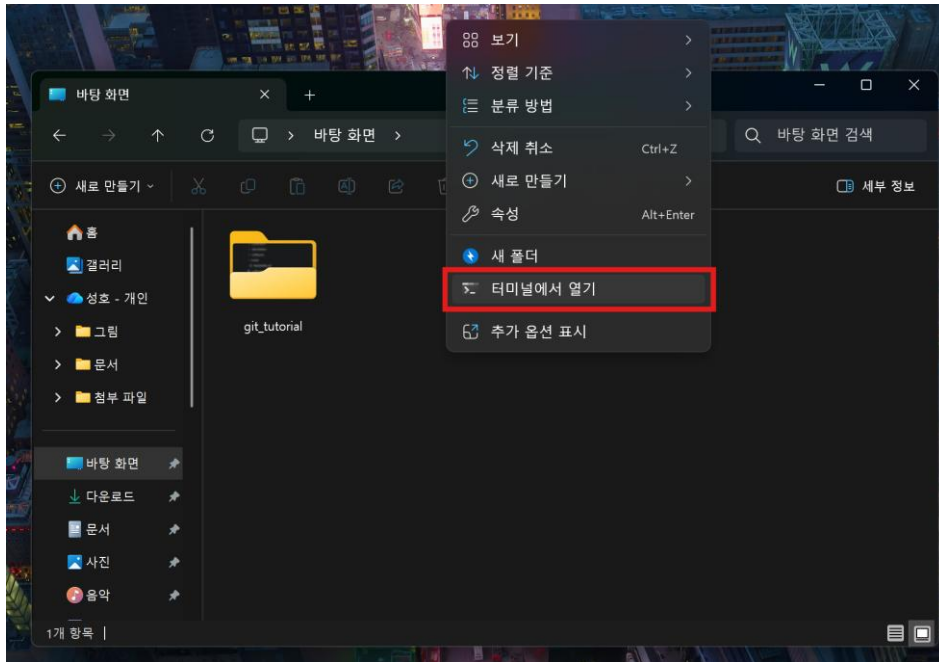
생성한 이슈의 번호를 확인합니다. 캡처에서는 #1 입니다.

2 작업 폴더에서 터미널 열기

저장소를 내려받을 작업 폴더를 파일 탐색기에서 엽니다. 아래 두 가지 방법 중 편한 방법으로 터미널을 실행합니다.

마우스 오른쪽 버튼으로 열기

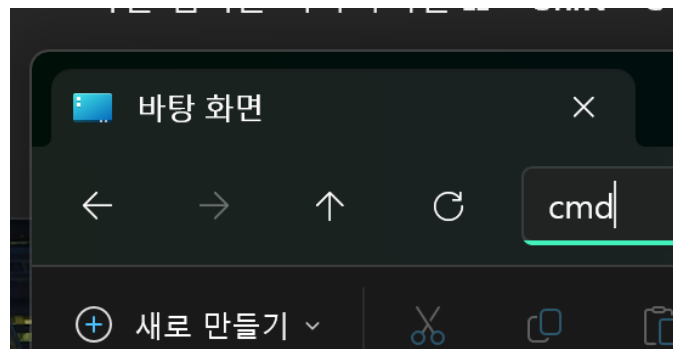
폴더의 빈 공간을 마우스 오른쪽 버튼으로 클릭하고 **터미널에서 열기**를 선택합니다.



저장소를 내려받을 위치에서 터미널을 엽니다.

팁 주소 표시줄에서 바로 열기

파일 탐색기의 주소 표시줄에 **cmd** 를 입력하고 **Enter** 를 누르면 현재 폴더에서 명령 프롬프트가 열립니다.

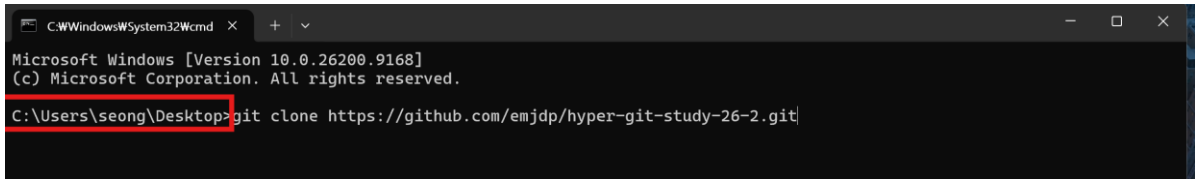


폴더 경로가 표시되는 주소 표시줄에 **cmd** 를 입력합니다.

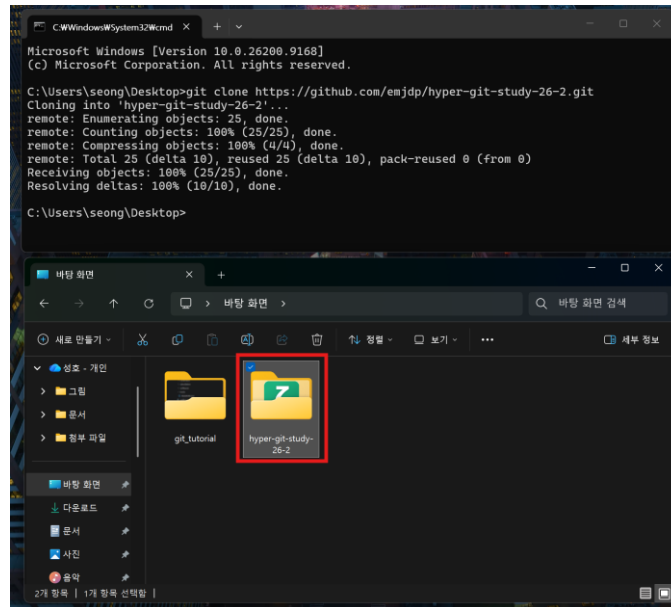
2 저장소 복제하고 VS Code 로 열기

1. 열린 터미널에 아래 명령어를 입력합니다.

```
git clone https://github.com/emjdp/hyper-git-study-26-2.git
```



명령어 앞에 표시되는 C:\...> 경로는 입력하지 않습니다.

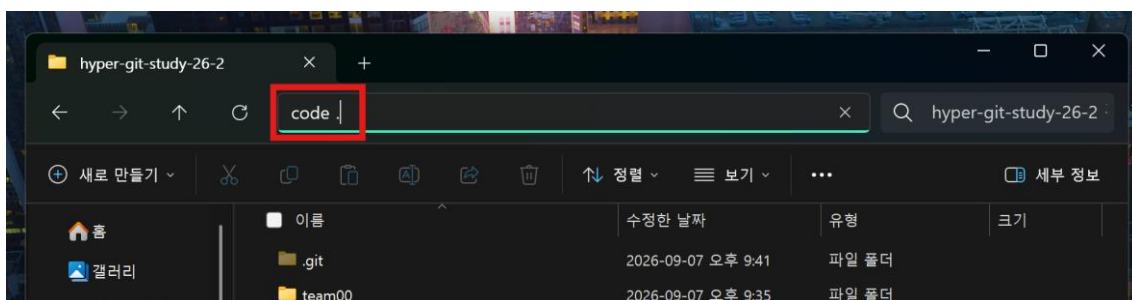


복제가 끝나면 hyper-git-study-26-2 폴더가 생깁니다.

2. 복제된 폴더로 이동해 VS Code 를 실행합니다.

```
cd hyper-git-study-26-2  
code .
```

팁 파일 탐색기에서 복제된 폴더를 연 뒤 주소 표시줄에 code .를 입력해도 됩니다. 점(.)은 현재 폴더를 뜻합니다.

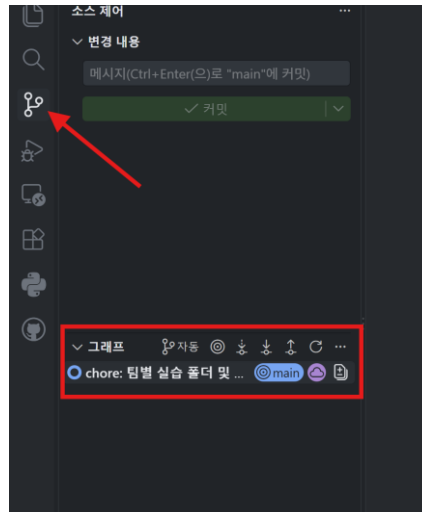


복제된 저장소 폴더 안에서 code .을 실행합니다.

code .가 실행되지 않으면 VS Code 의 **파일 → 폴더 열기**에서 복제된 폴더를 엽니다.

3 저장소 확인과 사용자 정보 설정

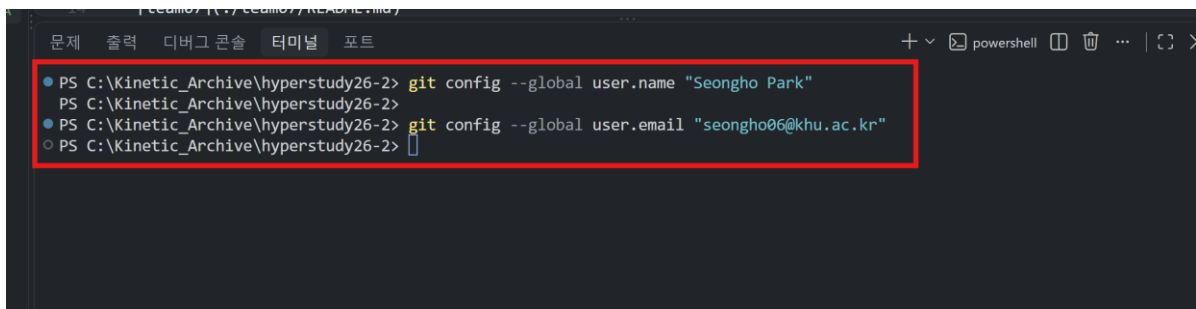
1. VS Code 왼쪽의 **소스 제어** 탭에서 저장소가 인식되는지 확인합니다.



소스 제어에 저장소의 커밋 그래프가 표시됩니다.

2. **보기** → **터미널**에서 다음 명령어를 실행합니다. 따옴표 안을 자신의 정보로 바꿉니다.

```
git config --global user.name "이름"
git config --global user.email "이메일"
```



VS Code 터미널에서 커밋에 기록할 이름과 이메일을 설정합니다.

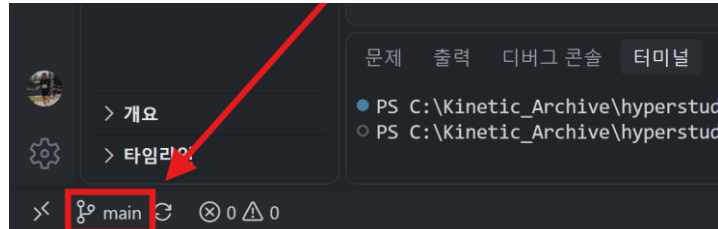
user.name 은 커밋에 표시할 이름입니다. user.email 에는 GitHub 에 등록한 이메일 또는 GitHub 의 noreply 이메일을 입력합니다. 공용 PC 에서 현재 저장소에만 적용하려면 --global 을 뺍니다.

설정 확인은 git config user.name 과 git config user.email 로 할 수 있습니다.

4 작업 브랜치 만들기

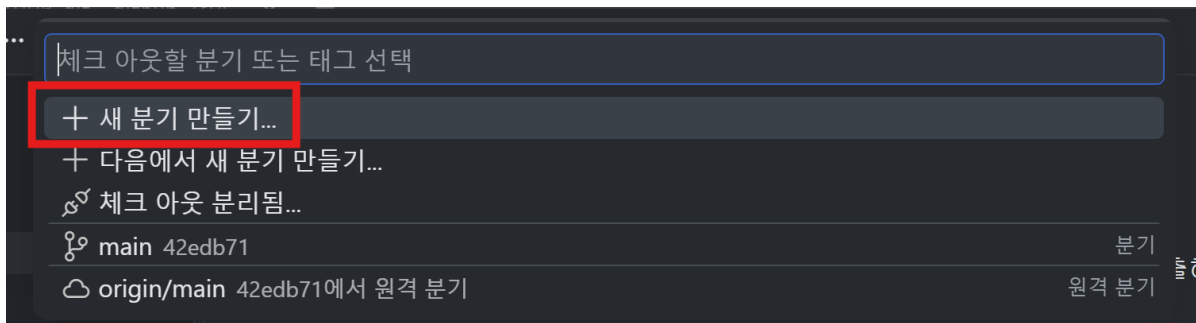
공동 저장소의 main 에서 바로 수정하지 않습니다. 각자 작업 브랜치에서 수정하고 PR 로 합칩니다.

1. VS Code 왼쪽 아래 브랜치가 main 인지 확인합니다. 기존에 복제한 저장소라면 **소스 제어** → ... → **풀(Pull)**로 최신 변경을 먼저 받습니다.



VS Code 왼쪽 아래에서 현재 브랜치를 찾습니다.

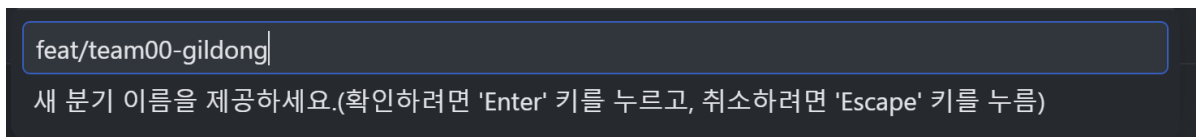
2. 왼쪽 아래의 main 을 클릭하고 **새 분기 만들기(Create new branch)**를 선택합니다.



현재 브랜치 이름을 클릭하면 브랜치 메뉴가 열립니다.

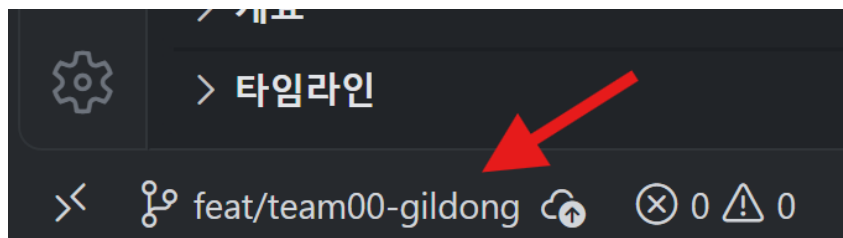
3. 다른 사람과 겹치지 않도록 팀, 이슈 번호, 이름을 포함해 입력합니다.

team00/issue-12-gildong



브랜치 이름을 입력하고 Enter 를 누릅니다. 캡처의 이름은 별도 예시입니다.

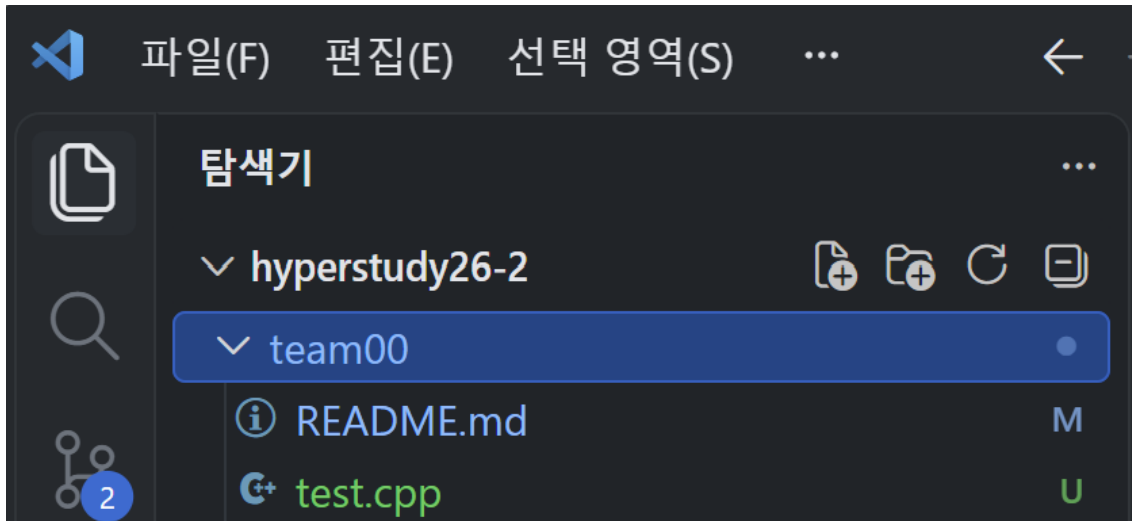
4. 왼쪽 아래 브랜치 이름이 방금 만든 이름으로 바뀌었는지 확인합니다.



작업 전 현재 브랜치를 확인합니다. 캡처의 브랜치 이름은 별도 예시입니다.

5 파일 수정하기

1. 자신의 팀 폴더 안에서 활동에 필요한 파일을 추가하거나 수정합니다. 다른 팀의 파일은 수정하지 않습니다.

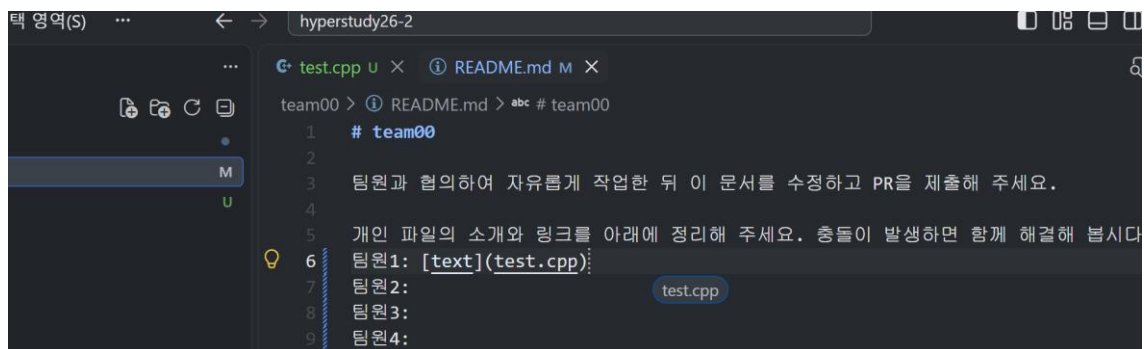


팀 활동에 필요한 파일을 만듭니다. 캡처에서는 test.cpp 를 사용했습니다.

2. 팀의 README.md 에 자신이 만든 파일을 링크합니다. 예를 들어 같은 폴더의 test.cpp 를 연결하려면 다음과 같이 작성합니다.

- [gildong의 실습 파일](./test.cpp)

팀 README 의 다른 사람 내용은 유지하고 자신의 항목을 추가하거나 수정합니다. **팁** VS Code 에서 파일을 편집기로 끌어온 뒤 **Shift** 를 누른 상태로 놓아도 링크를 삽입할 수 있습니다.



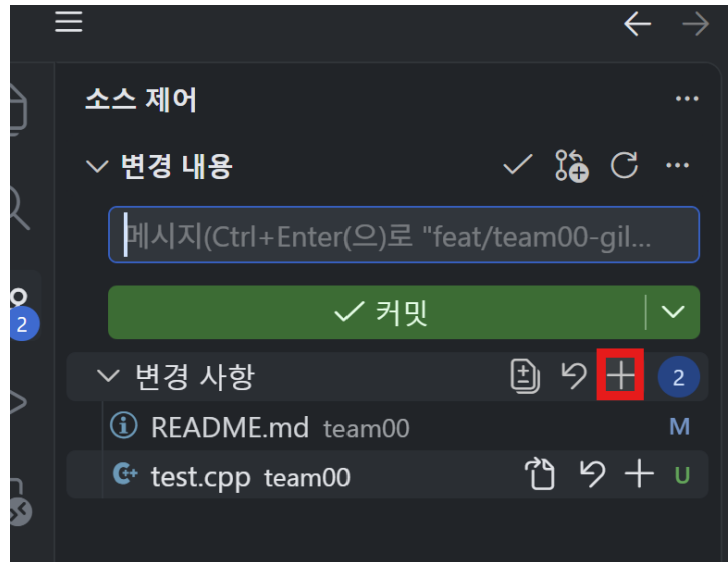
README 에 팀 활동 파일의 링크를 추가합니다.

3. 파일을 저장하고 README 미리보기에서 링크가 올바른 파일을 가리키는지 확인합니다. 미리보기는 **Ctrl+Shift+V** 로 엽니다.

변경 표시 U 새 파일 · M 수정된 파일 · A 스테이징된 새 파일 · D 삭제된 파일 · R 이름이 바뀐 파일

6 변경 사항 커밋하기

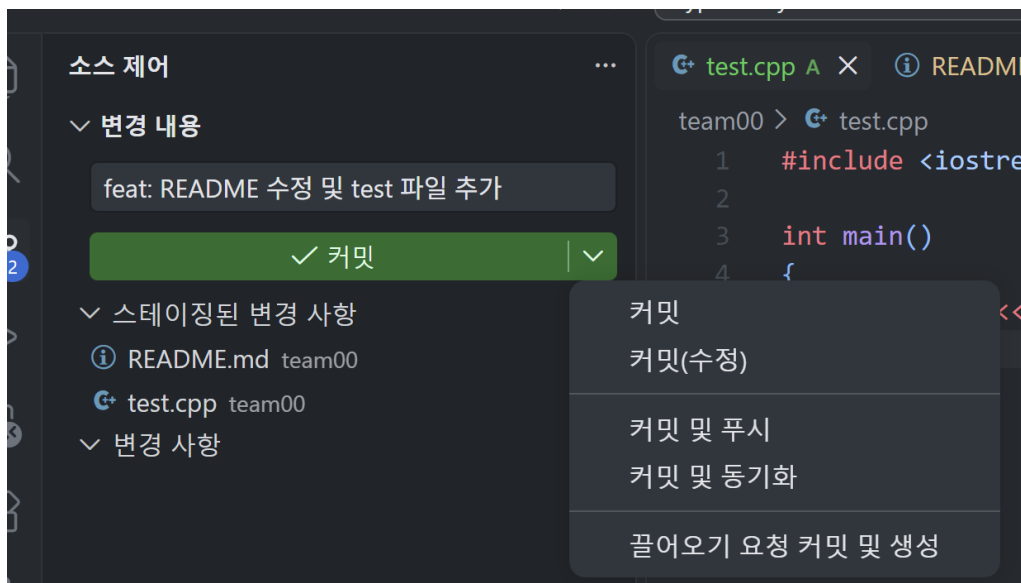
1. 소스 제어에서 변경된 파일을 열어 내용을 확인합니다.
2. 이번 커밋에 넣을 파일 옆의 +를 눌러 스테이징합니다. 모든 변경을 넣으려면 **변경 내용** 오른쪽의 +를 누릅니다.



스테이징한 파일만 다음 커밋에 포함됩니다.

3. 커밋 메시지를 입력하고 **커밋(Commit)**을 누릅니다.

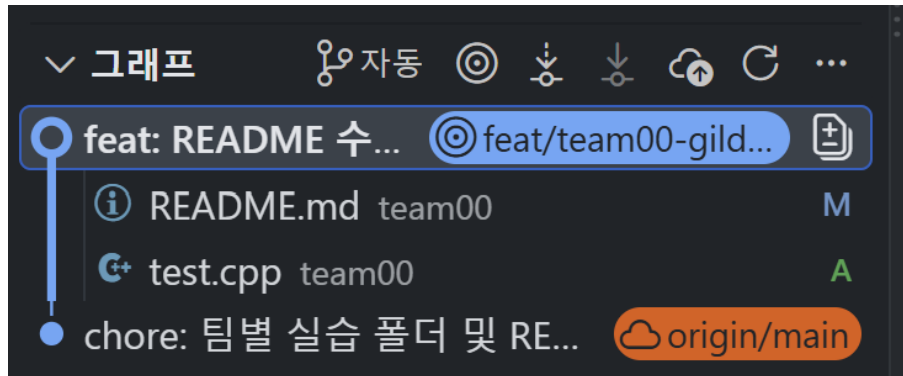
docs: team00 활동 내용과 README 링크 추가



커밋 메시지를 입력한 뒤 커밋을 실행합니다.

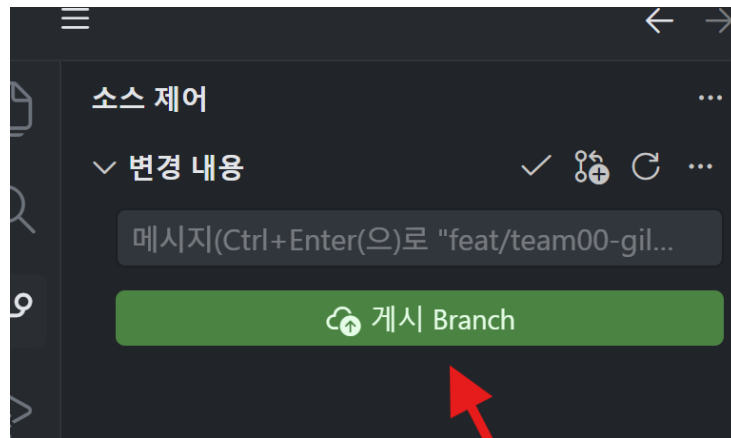
6 작업 브랜치를 GitHub에 게시하기

커밋 직후의 변경 내용은 아직 내 컴퓨터에만 저장되어 있습니다.

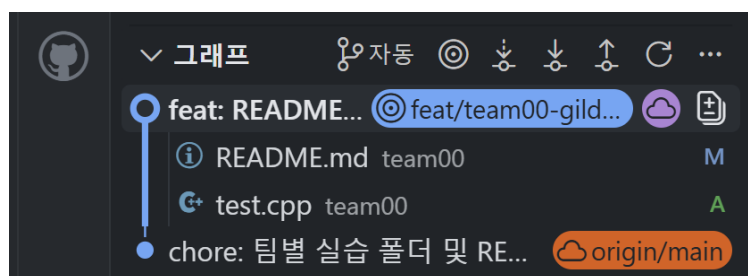


작업 브랜치에 새 커밋이 생겼지만 원격 main에는 아직 없습니다.

1. 첫 푸시는 **분기 게시(Publish Branch)**를 누릅니다. 이후 커밋은 **소스 제어** → ... → **푸시(Push)**로 올립니다. 인증 화면이 나오면 초대를 수락한 GitHub 계정으로 로그인합니다.



분기 게시를 눌러 작업 브랜치를 원격 저장소에 올립니다.



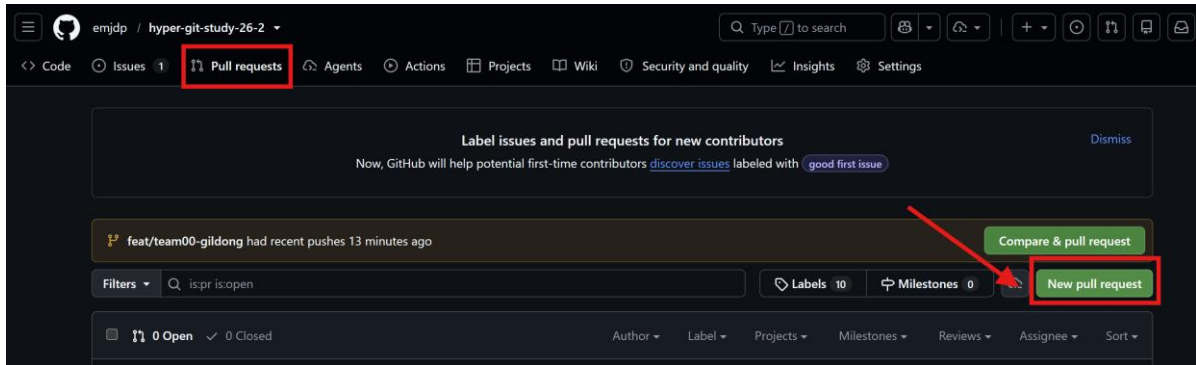
게시 후 그래프에서 작업 브랜치의 원격 표시를 확인합니다.

2. GitHub 저장소에서 자신의 브랜치를 선택하고 수정한 파일과 커밋이 보이는지 확인합니다. 변경 목록이 비어 있는 것만으로는 푸시 완료를 알 수 없습니다.

7 GitHub 에서 Pull Request 만들기

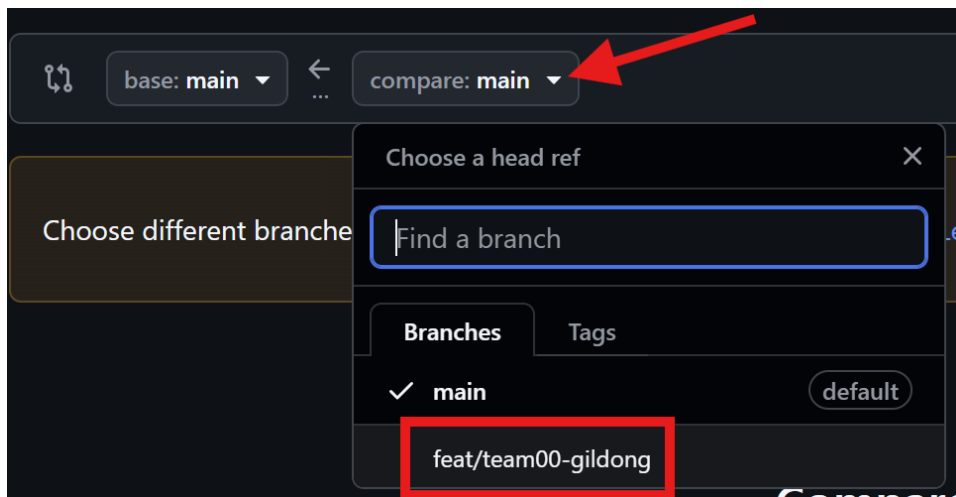
PR 은 작업 브랜치의 변경 내용을 main 에 합쳐 달라는 요청입니다. GitHub 웹 또는 뒤에 나오는 VS Code 중 한 가지 방법만 사용합니다.

1. 푸시 후 저장소에 표시되는 **Compare & pull request**를 누릅니다. 보이지 않으면 **Pull requests** → **New pull request**로 들어갑니다.

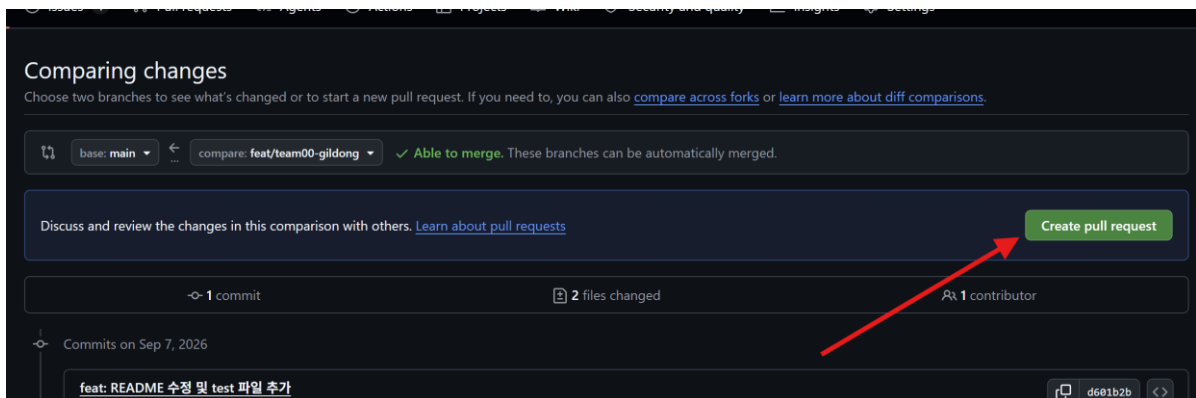


푸시한 브랜치에서 PR 을 시작합니다.

2. **base** 는 main, **compare** 는 자신의 작업 브랜치인지 확인합니다.



변경을 받을 브랜치와 변경을 보낼 브랜치를 확인합니다.



변경 내용을 확인하고 Create pull request 를 눌러 작성 화면으로 이동합니다.

7 PR 내용과 담당자 작성하기

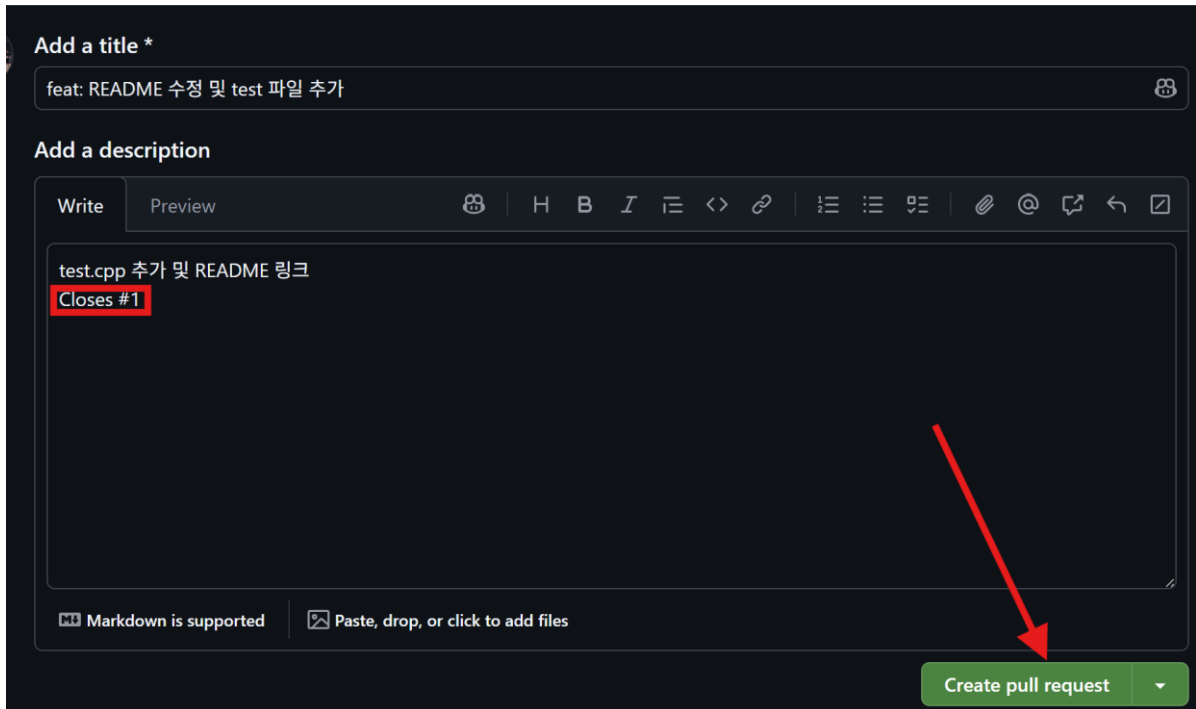
3. 제목과 본문을 작성합니다. #12 는 앞에서 만든 이슈 번호로 바꿉니다.

제목: [team00] 활동 파일과 README 링크 추가

본문:

- 팀 활동 파일을 추가했습니다.
- README 링크가 올바른 파일을 가리키는지 확인했습니다.

Closes #12



Add a title *

feat: README 수정 및 test 파일 추가

Add a description

Write Preview

test.cpp 추가 및 README 링크
Closes #1

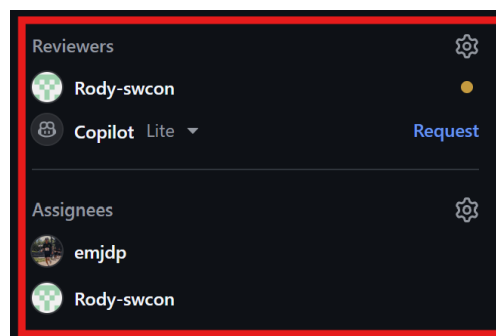
Markdown is supported Paste, drop, or click to add files

Create pull request

제목과 본문을 작성하고 Closes 문구로 이슈를 연결합니다.

4. **Create pull request** 를 누릅니다. PR 의 **Assignees** 에는 자신을, **Reviewers** 에는 팀원을 지정합니다.

Closes #12 를 본문에 넣으면 main 에 머지할 때 해당 이슈가 자동으로 닫힙니다.



Reviewers

Rody-swcon

Copilot Lite Request

Assignees

emjdp

Rody-swcon

Reviewers 는 검토할 팀원, Assignees 는 작업 담당자인 자신입니다.

7 VS Code 에서 PR 만들기

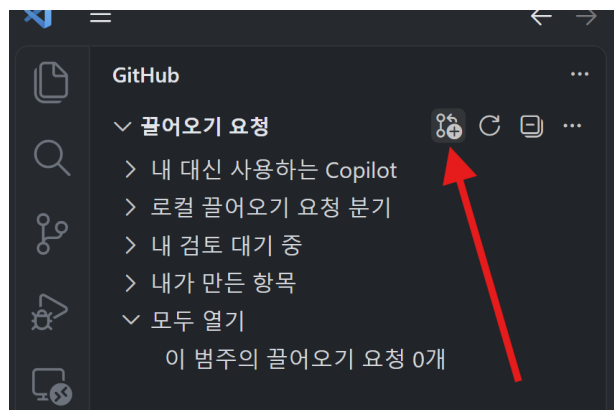
GitHub 웹에서 이미 PR 을 만들었다면 뒤의 리뷰 단계로 넘어갑니다.

1. 확장 탭에서 **GitHub Pull Requests** 를 설치하고 GitHub 계정으로 로그인합니다.



GitHub Pull Requests 확장을 설치합니다.

2. 작업 브랜치를 게시한 상태에서 **Create Pull Request** 아이콘을 선택합니다.

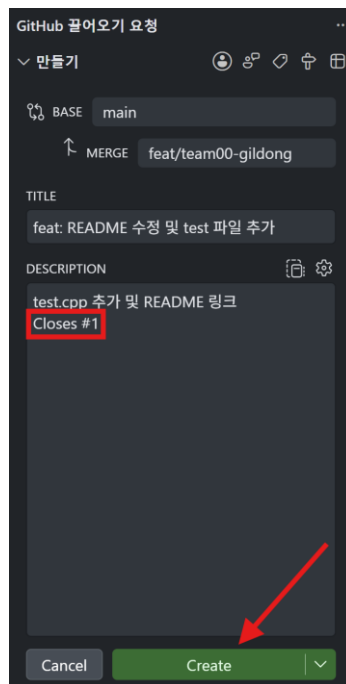


Pull Requests 보기 위쪽의 Create Pull Request 아이콘입니다.

메뉴가 보이지 않으면 명령 팔레트에서 GitHub Pull Requests: Create Pull Request 를 검색합니다.

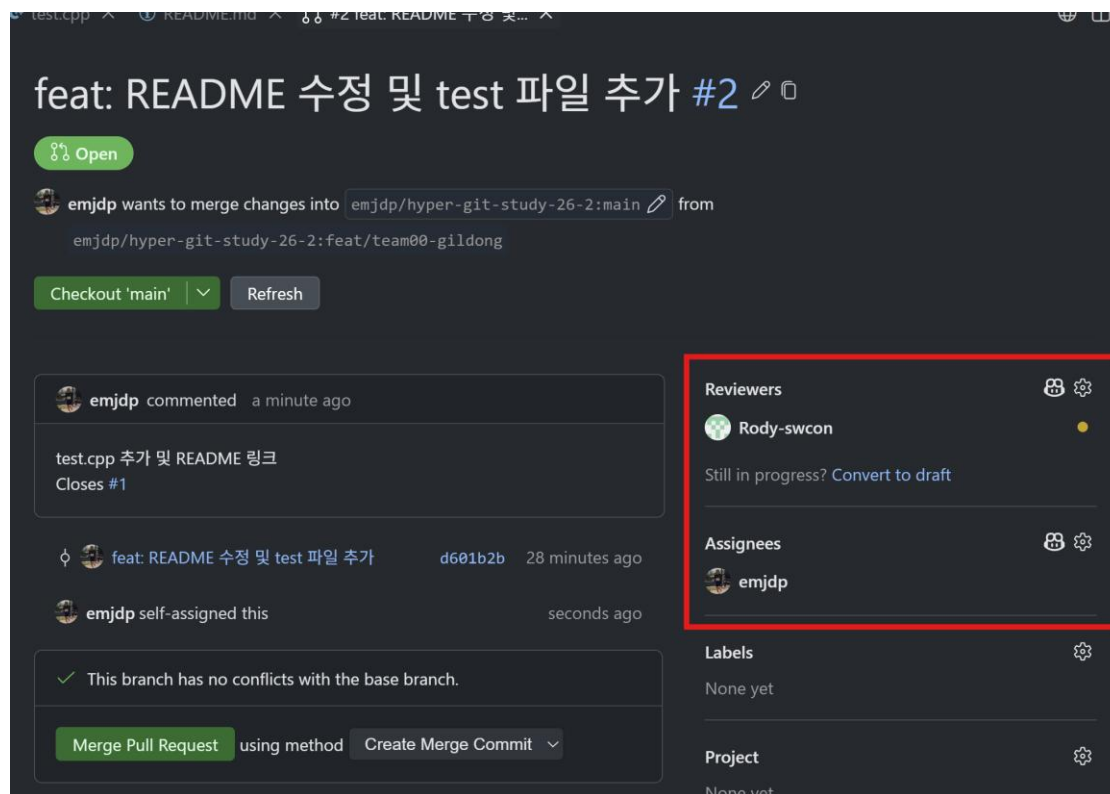
7 VS Code 에서 PR 내용 작성하기

3. 기준 브랜치는 main, 비교 브랜치는 자신의 작업 브랜치로 선택합니다. 제목과 본문을 적고 Closes #자신의이슈번호를 넣은 뒤 **Create** 를 누릅니다.



브랜치 방향과 Closes 문구를 확인하고 PR 을 생성합니다.

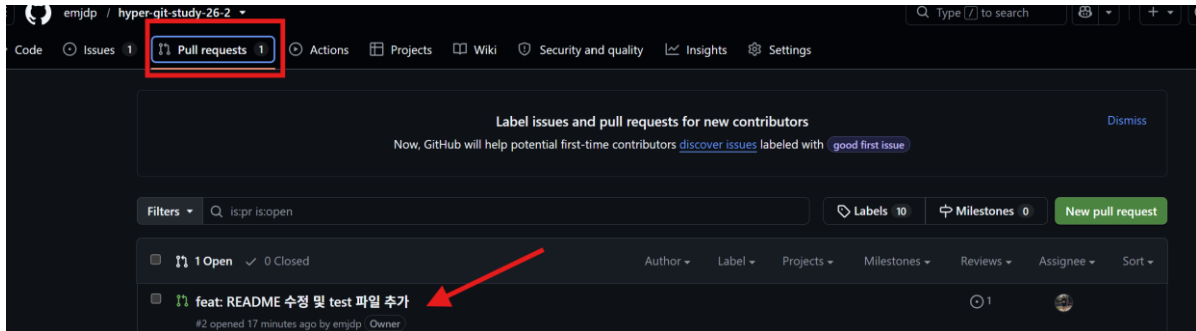
4. 생성된 PR 에서 담당자는 자신, 리뷰어는 팀원으로 지정합니다. GitHub 웹에서 지정해도 됩니다.



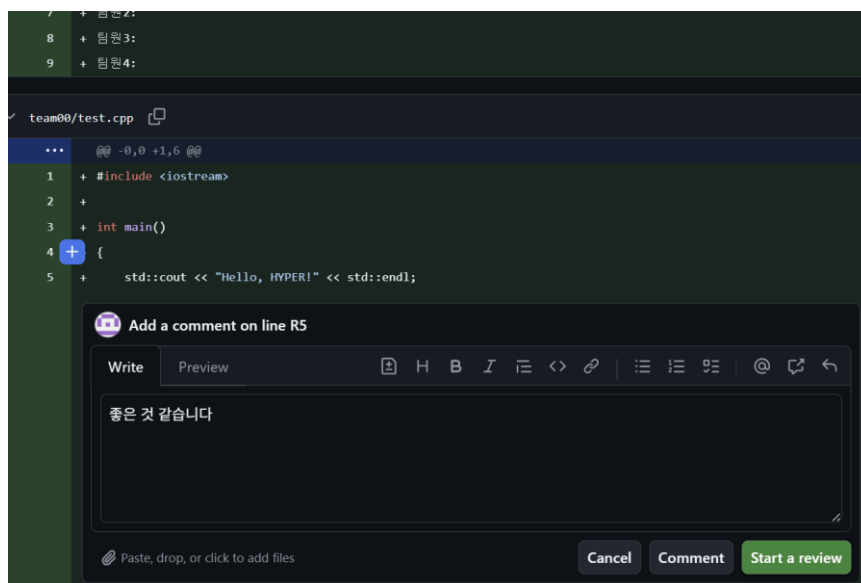
생성한 PR 의 이슈 연결 문구와 Reviewers 및 Assignees 를 확인합니다.

8 팀원의 PR 리뷰하기

1. 팀원의 PR 을 열고 **Files changed** 탭에서 변경 내용을 확인합니다. 파일 내용과 README 링크를 함께 살펴봅니다.



Pull requests 목록에서 팀원의 PR 을 선택합니다.



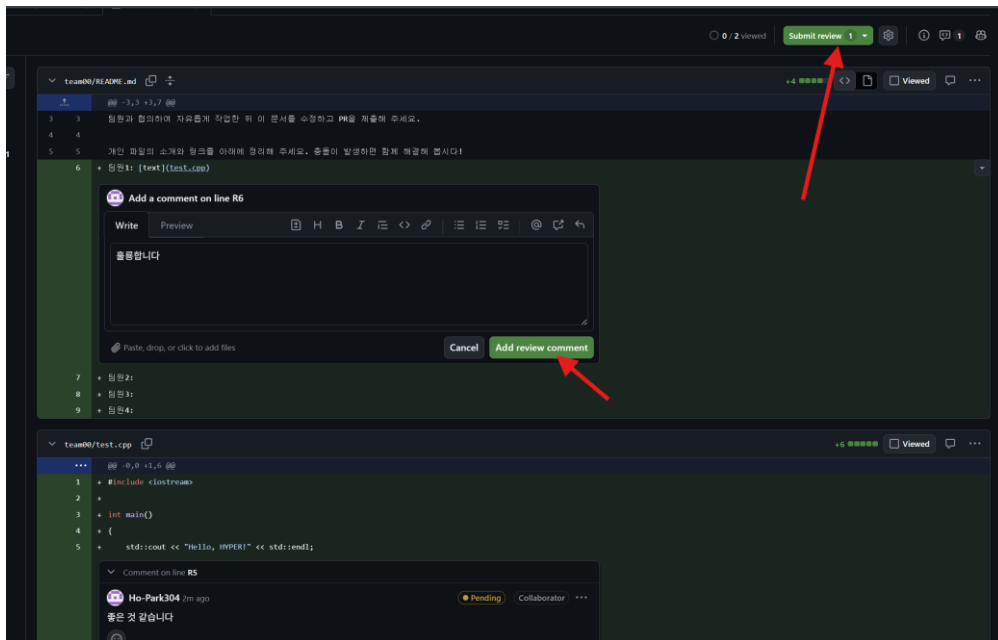
변경된 파일과 줄을 확인하고 필요한 곳에 의견을 남깁니다.

2. 의견을 남길 줄 옆에 마우스를 올려 +를 누르고 댓글을 작성합니다.

Add single comment 는 즉시 등록합니다. **Start a review** 는 댓글을 임시로 모아 두므로 마지막에 리뷰를 제출해야 상대방에게 전달됩니다.

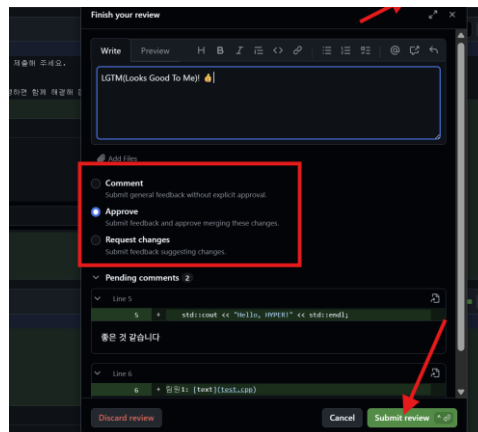
8 리뷰 제출과 수정 반영하기

Start a review 로 시작했다면 댓글이 Pending 상태로 남아 있습니다. 검토를 끝내고 리뷰를 제출해야 상대방에게 전달됩니다.



의견을 추가한 뒤 화면 위의 리뷰 제출 버튼으로 이동합니다.

3. **Review changes** 또는 **Submit review** 를 눌러 검토 결과를 선택하고 제출합니다.



검토 결과를 선택한 뒤 Submit review 로 제출합니다.

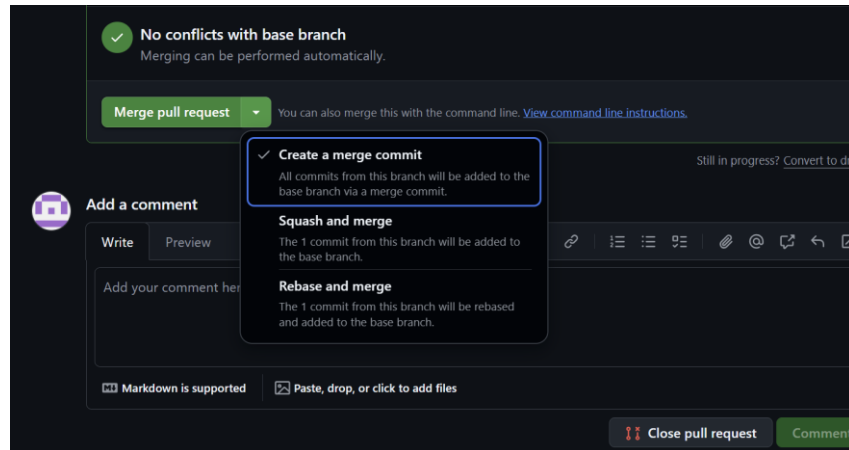
Comment 일반 의견이나 질문 · **Approve** 승인 · **Request changes** 머지 전 수정 요청

4. 수정 요청을 받았다면 **같은 작업 브랜치**에서 수정하고 커밋과 푸시를 다시 합니다. 기존 PR 에 자동으로 반영되므로 새 PR 을 만들 필요가 없습니다. 수정 내용을 알리고 다시 검토를 요청합니다.

9 PR 머지하기

리뷰와 수정이 끝나면 PR의 변경 내용을 main에 합칩니다. 이번 실습에서는 **Squash and merge**를 사용합니다.

1. PR의 대화 화면에서 머지 버튼 옆 메뉴를 열고 **Squash and merge**를 선택합니다.



Squash and merge를 선택해 실습 중 만든 여러 커밋을 하나로 합칩니다.

2. 최종 커밋 메시지를 확인하고 **Confirm squash and merge**를 누릅니다.

3. PR이 **Merged**로 표시되는지 확인합니다. 저장소의 main에서 파일이 반영되었는지, 연결한 이슈가 **Closed**로 바뀌었는지도 확인합니다.

4. 필요하면 **Delete branch**로 원격 작업 브랜치를 삭제합니다.

머지 방식의 차이

Create a merge commit은 개별 커밋을 유지하고 두 브랜치를 합친 머지 커밋을 추가합니다.

Squash and merge는 PR의 여러 커밋을 하나로 합쳐 기존 브랜치에 추가합니다.

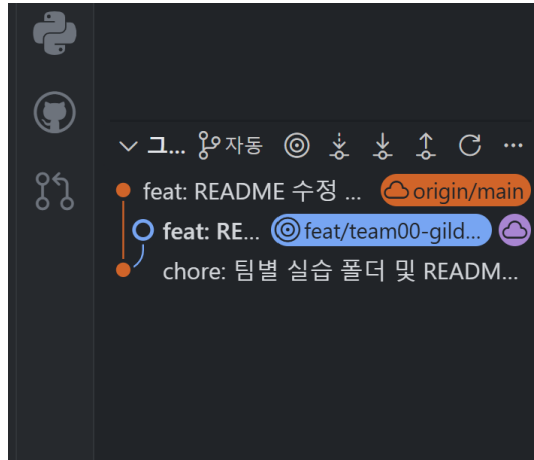
Rebase and merge는 머지 커밋 없이 개별 커밋을 기존 브랜치 끝에 차례로 다시 적용합니다.

머지가 막히거나 충돌이 생겼을 때

승인이나 검사가 필요하면 PR에 표시된 조건을 먼저 충족합니다. 충돌이 있다면 작업 브랜치에서 폐치한 뒤, 명령 팔레트의 **Git: Merge Branch**로 origin/main을 합칩니다. **Merge Changes**의 파일에서 내 변경과 팀원의 변경을 확인해 정리하고 충돌 표시를 제거합니다. 저장 → 스테이징 → 커밋 → 푸시 후 PR을 다시 확인합니다.

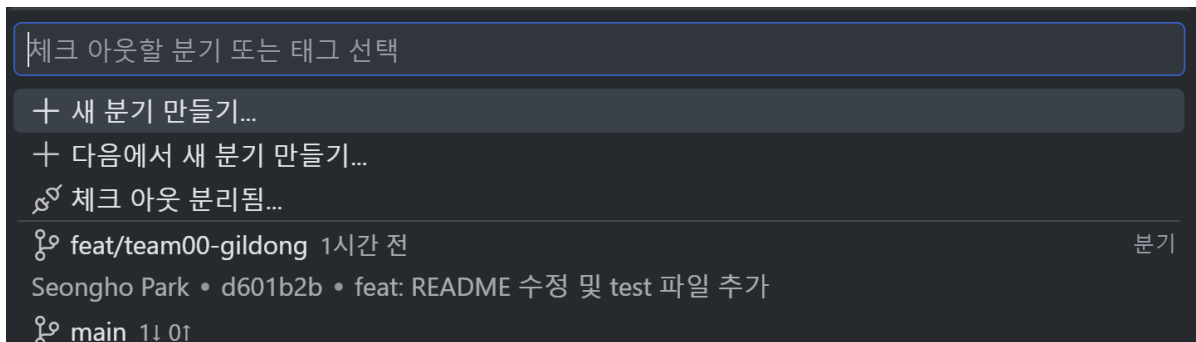
10 로컬 main 으로 전환하기

GitHub 에서 머지해도 내 컴퓨터의 main 은 자동으로 갱신되지 않습니다.



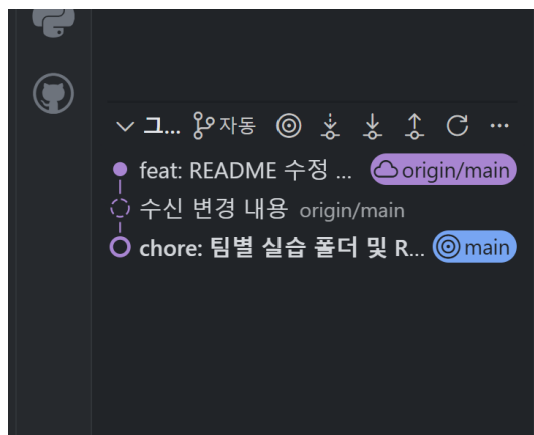
VS Code 그래프에서도 원격 브랜치와 커밋 위치를 확인할 수 있습니다.

1. VS Code 왼쪽 아래 브랜치 이름을 클릭해 **로컬 main** 으로 전환합니다. 목록에서는 origin/main 이 아닌 main 을 선택합니다.



브랜치 목록에서 main 항목을 선택합니다.

2. 원격 정보가 갱신되어 있다면 내려받을 커밋 수(↓)가 표시됩니다.

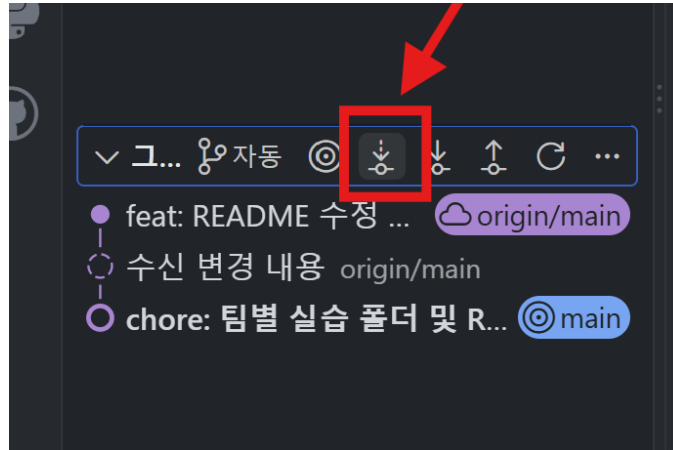


main 과 origin/main 의 위치가 다르면 아직 받을 변경 내용이 있습니다.

10 페치와 풀로 최신 변경 받기

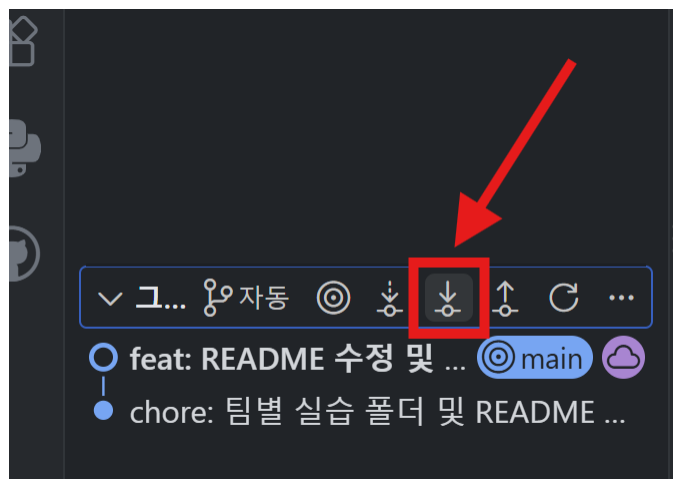
GitHub 에서 머지해도 내 컴퓨터의 main 은 자동으로 갱신되지 않습니다.

3. 소스 제어 → ... → 모든 원격에서 페치(Fetch from All Remotes)를 선택합니다. 페치는 원격의 최신 정보를 가져오며 파일 내용은 아직 바꾸지 않습니다.



페치로 원격 저장소의 최신 커밋 정보를 가져옵니다.

4. 같은 메뉴의 풀(Pull)로 원격 main 의 변경 내용을 로컬에 반영합니다.



현재 브랜치가 main 인지 확인하고 Pull 을 실행합니다.

5. 팀원의 파일과 내 파일이 보이는지 확인합니다. 다음 작업은 최신 main 에서 **새 브랜치**를 만들어 시작합니다.

실습 완료 PR 이 Merged 이고, 이슈가 Closed 이며, 로컬 main 에도 최종 파일이 반영되어 있으면 전체 협업 과정을 마친 것입니다.